PYTHON ACTIVITY-BASED LEARNING

10 Beginner-Friendly Case Studies

Complete documentation: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python code, test cases, sample output, and conclusion.

Repository name: PythonActivity-Based-Learning

Thota Rajitha

26001254 / CSM / 06

Selected Case Studies

1. Case 1: University Scholarship Allocation
2. Case 7: Retail Billing with Multiple Discount Rules
3. Case 12: Attendance Recovery Computation
4. Case 14: Warehouse Inventory Reconciliation
5. Case 17: Student Performance Trend Analysis
6. Case 27: Advanced Numeric Pattern Generator
7. Case 29: Password Policy Validation Engine
8. Case 42: Student Marks Matrix Analysis
9. Case 49: Tuple-Based Product Catalogue Audit
10. Case 53: Food Waste Monitoring System

Case Study 1: University Scholarship Allocation

1. Problem Statement

Create a program that checks whether a student qualifies for a scholarship using aggregate marks, minimum marks, attendance, and family income.

2. Objectives

• Use variables, arithmetic operators, and conditional statements.
• Check several eligibility conditions together.
• Display a clear eligible/not-eligible result.

3. Assumptions

• Marks are percentages from 0 to 100.
• Attendance is a percentage from 0 to 100.
• Scholarship rules: aggregate at least 80%, each required subject at least 60%, attendance at least 75%, and annual family income no more than ■250,000.

4. Algorithm

1. Start.
2. Read aggregate marks, minimum subject mark, attendance, and annual family income.
3. Validate that percentages are between 0 and 100 and income is non-negative.
4. Check all scholarship conditions.
5. Display eligibility and any failed conditions.
6. Stop.

5. Flowchart

Start → Input marks, attendance, income → Validate input → Check all criteria → Display result → End

6. Pseudocode

READ aggregate, minimum_subject, attendance, income\nIF any percentage is outside 0..100 OR income < 0\n DISPLAY invalid input\nELSE\n IF aggregate >= 80 AND minimum_subject >= 60 AND attendance >= 75 AND income <= 250000\n DISPLAY eligible\n ELSE\n DISPLAY not eligible and failed criteria\nSTOP

7. Python Source Code

```
aggregate = float(input("Aggregate marks (%): "))
minimum_subject = float(input("Lowest subject mark (%): "))
attendance = float(input("Attendance (%): "))
income = float(input("Annual family income (■): "))

if not (0 <= aggregate <= 100 and 0 <= minimum_subject <= 100
        and 0 <= attendance <= 100 and income >= 0):
    print("Invalid input. Check the values entered.")
else:
    failed = []
    if aggregate < 80:
        failed.append("aggregate marks below 80%")
    if minimum_subject < 60:
        failed.append("a subject mark below 60%")
    if attendance < 75:
        failed.append("attendance below 75%")
    if income > 250000:
        failed.append("family income above ■250000")

    if len(failed) == 0:
        print("Result: Eligible for scholarship")
    else:
        print("Result: Not eligible for scholarship")
        for reason in failed:
            print("-", reason)
```

8. Test Cases

Input	Expected check	Expected result
85	70	90
78	65	80

Input	Expected check	Expected result
90	55	90

9. Sample Output Screenshot / Transcript

```
Aggregate marks (%): 85
Lowest subject mark (%): 70
Attendance (%): 90
Annual family income (■): 200000
Result: Eligible for scholarship
```

10. Conclusion

The program applies multiple conditions to a student record and explains why an applicant does not qualify.

Case Study 7: Retail Billing with Multiple Discount Rules

1. Problem Statement

Prepare a simple bill for several products, apply a discount when the subtotal reaches a threshold, add tax, and print the final amount.

2. Objectives

• Use loops to process several products.
• Calculate subtotal, discount, tax, and final bill.
• Use conditional statements to apply a discount rule.

3. Assumptions

• Enter a non-negative price and a positive quantity for each item.
• A 10% discount applies when subtotal is at least ■2,000.
• Tax is 5% of the amount after discount.

4. Algorithm

1. Start.
2. Read the number of products.
3. For each product, read price and quantity and add price × quantity to subtotal.
4. Apply 10% discount if subtotal is at least ■2,000.
5. Calculate 5% tax after discount.
6. Display subtotal, discount, tax, and final bill.
7. Stop.

5. Flowchart

Start → Read number of items → Loop through price and quantity → Calculate subtotal → Apply discount rule → Calculate tax and total → Display bill → End

6. Pseudocode

READ number_of_items\nsubtotal = 0\nREPEAT for each item\n READ price, quantity\n subtotal = subtotal + price * quantity\nIF subtotal >= 2000 THEN discount = subtotal * 0.10 ELSE discount = 0\ntaxable = subtotal - discount\ntax = taxable * 0.05\nDISPLAY subtotal, discount, tax, taxable + tax\nSTOP

7. Python Source Code

```
count = int(input("Number of products: "))
subtotal = 0.0

if count <= 0:
    print("Enter at least one product.")
else:
    for i in range(1, count + 1):
        price = float(input("Price of product " + str(i) + " (■): "))
        quantity = int(input("Quantity of product " + str(i) + ": "))
        if price < 0 or quantity <= 0:
            print("Invalid price or quantity.")
            break
        subtotal += price * quantity
    else:
        if subtotal >= 2000:
            discount = subtotal * 0.10
        else:
            discount = 0
        taxable_amount = subtotal - discount
        tax = taxable_amount * 0.05
        total = taxable_amount + tax
        print("Subtotal: ■{:.2f}".format(subtotal))
        print("Discount: ■{:.2f}".format(discount))
        print("Tax: ■{:.2f}".format(tax))
        print("Final bill: ■{:.2f}".format(total))
```

8. Test Cases

Input	Expected check	Expected result
2 items: 1000×1, 1200×1	subtotal 2200	discount ■220.00; tax ■99.00; final ■2079.00
1 item: 500×2	subtotal 1000	discount ■0.00; tax ■50.00; final ■1050.00

9. Sample Output Screenshot / Transcript

```
Number of products: 2
Price of product 1 (■): 1000
Quantity of product 1: 1
Price of product 2 (■): 1200
Quantity of product 2: 1
Subtotal: ■2200.00
Discount: ■220.00
Tax: ■99.00
Final bill: ■2079.00
```

10. Conclusion

The program demonstrates loop-based data entry and applies a threshold discount before calculating tax.

Case Study 12: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance percentage and find the minimum number of consecutive classes a student must attend to reach the required attendance percentage.

2. Objectives

• Calculate percentages using arithmetic.
• Use a loop to find the minimum future classes needed.
• Apply input validation and a target condition.

3. Assumptions

• Classes attended cannot exceed classes conducted.
• The student attends every future class.
• The eligibility target is 75%.

4. Algorithm

1. Start.
2. Read classes conducted and classes attended.
3. Validate the values.
4. Calculate current attendance percentage.
5. If the target is not reached, add one attended class and one conducted class repeatedly until the target is reached.
6. Display the current percentage and minimum classes needed.
7. Stop.

5. Flowchart

Start → Read attended and conducted → Validate → Calculate percentage → Target reached? → If no, loop adding attended classes → Display answer → End

6. Pseudocode

READ attended, conducted\nIF conducted <= 0 OR attended < 0 OR attended > conducted: DISPLAY invalid\nELSE\n percentage = attended / conducted * 100\n extra = 0\n WHILE (attended + extra) / (conducted + extra) * 100 < 75\n extra = extra + 1\n DISPLAY percentage and extra\nSTOP

7. Python Source Code

```
attended = int(input("Classes attended: "))
conducted = int(input("Total classes conducted: "))
target = 75

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance details.")
else:
    current = attended / conducted * 100
    extra = 0
    while (attended + extra) / (conducted + extra) * 100 < target:
        extra += 1
    print("Current attendance: {:.2f}%".format(current))
    if extra == 0:
        print("Target already achieved.")
    else:
        print("Additional consecutive classes to attend:", extra)
```

8. Test Cases

Input	Expected check	Expected result
attended=60, conducted=100	Current 60.00%	Additional classes: 60
attended=80, conducted=100	Current 80.00%	Target already achieved

9. Sample Output Screenshot / Transcript

```
Classes attended: 60
Total classes conducted: 100
Current attendance: 60.00%
Additional consecutive classes to attend: 60
```

10. Conclusion

The loop searches for the smallest number of future classes that reaches the attendance threshold.

Case Study 14: Warehouse Inventory Reconciliation

1. Problem Statement

For each product, calculate expected closing stock from opening stock, incoming stock, and dispatched stock; compare it with the physically counted stock and report discrepancies.

2. Objectives

• Use lists to store product information.
• Calculate expected stock using arithmetic.
• Use conditions to report shortages, surpluses, or matching stock.

3. Assumptions

• All quantities are non-negative whole numbers.
• Expected closing stock = opening stock + incoming stock – dispatched stock.
• Physical stock is the counted quantity.

4. Algorithm

1. Start.
2. Read the number of products.
3. For each product, read name, opening stock, incoming quantity, dispatched quantity, and physical stock.
4. Calculate expected closing stock.
5. Compare expected and physical stock.
6. Report shortage, surplus, or no discrepancy.
7. Stop.

5. Flowchart

Start → Read product count → Loop: read stock details → Calculate expected stock → Compare physical stock → Print discrepancy → End

6. Pseudocode

READ n\nREPEAT n times\n READ name, opening, incoming, dispatched, physical\n expected = opening + incoming - dispatched\n difference = physical - expected\n IF difference < 0: DISPLAY shortage of -difference\n ELSE IF difference > 0: DISPLAY surplus of difference\n ELSE: DISPLAY stock matches\nSTOP

7. Python Source Code

```
n = int(input("Number of products: "))

if n <= 0:
    print("Enter a positive number of products.")
else:
    for i in range(1, n + 1):
        name = input("Product name: ")
        opening = int(input("Opening stock: "))
        incoming = int(input("Incoming stock: "))
        dispatched = int(input("Dispatched stock: "))
        physical = int(input("Physical closing stock: "))

        if min(opening, incoming, dispatched, physical) < 0 or dispatched > opening + incoming:
            print("Invalid stock values for", name)
            continue

        expected = opening + incoming - dispatched
        difference = physical - expected
        print("Expected closing stock:", expected)
        if difference < 0:
            print("Shortage:", -difference)
        elif difference > 0:
            print("Surplus:", difference)
        else:
            print("Stock matches.")
```

8. Test Cases

Input	Expected check	Expected result
opening 50, incoming 20, dispatched 30, physical 38	expected 40	Shortage: 2
opening 10, incoming 5, dispatched 3, physical 12	expected 12	Stock matches.

9. Sample Output Screenshot / Transcript

```
Number of products: 1
Product name: Pens
Opening stock: 50
Incoming stock: 20
Dispatched stock: 30
Physical closing stock: 38
Expected closing stock: 40
Shortage: 2
```

10. Conclusion

The program reconciles recorded stock with physical stock and makes differences easy to identify.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement

Compare a student's marks across three assessments for multiple subjects, calculate changes, and identify subjects with declining performance.

2. Objectives

• Store subject marks in lists.
• Calculate improvement or decline between assessments.
• Use loops and conditions to classify trends.

3. Assumptions

• Each mark is from 0 to 100.
• Three assessment marks are entered for every subject.
• A lower third-assessment mark than first-assessment mark is treated as a decline.

4. Algorithm

1. Start.
2. Read the number of subjects.
3. For each subject, read its name and marks in three assessments.
4. Validate marks.
5. Calculate change from first to third assessment.
6. Classify as improved, declined, or unchanged.
7. Display the trend for each subject.
8. Stop.

5. Flowchart

Start → Read subject count → Loop through subjects and three marks → Calculate change → Classify trend → Display → End

6. Pseudocode

READ n\nREPEAT n times\n READ subject, mark1, mark2, mark3\n change = mark3 - mark1\n IF change > 0: trend = Improved\n ELSE IF change < 0: trend = Declined\n ELSE: trend = Unchanged\n DISPLAY subject, change, trend\nSTOP

7. Python Source Code

```
n = int(input("Number of subjects: "))

if n <= 0:
    print("Enter at least one subject.")
else:
    for i in range(n):
        subject = input("Subject name: ")
        first = float(input("Assessment 1 mark: "))
        second = float(input("Assessment 2 mark: "))
        third = float(input("Assessment 3 mark: "))

        if not (0 <= first <= 100 and 0 <= second <= 100 and 0 <= third <= 100):
            print("Invalid marks. Marks must be from 0 to 100.")
            continue

        change = third - first
        if change > 0:
            trend = "Improved"
        elif change < 0:
            trend = "Declined"
        else:
            trend = "Unchanged"

        print(subject, "- change:", "{:+.1f}".format(change), "marks;", trend)
```

8. Test Cases

Input	Expected check	Expected result
Maths: 60, 65, 75	change +15	Improved
English: 80, 75, 70	change -10	Declined
Science: 70, 72, 70	change 0	Unchanged

9. Sample Output Screenshot / Transcript

```
Number of subjects: 2
Subject name: Maths
Assessment 1 mark: 60
Assessment 2 mark: 65
Assessment 3 mark: 75
Maths - change: +15.0 marks; Improved
Subject name: English
Assessment 1 mark: 80
Assessment 2 mark: 75
Assessment 3 mark: 70
English - change: -10.0 marks; Declined
```

10. Conclusion

The program compares assessment results and labels each subject's trend using simple conditions.

Case Study 27: Advanced Numeric Pattern Generator

1. Problem Statement

Generate a centered numeric pyramid for a positive number of rows, rejecting invalid dimensions.

2. Objectives

• Use nested loops for pattern generation.
• Practise counters and string repetition.
• Validate user input.

3. Assumptions

• The number of rows must be between 1 and 9 so each row remains a simple sequence of digits.
• Row i prints numbers from 1 through i, then numbers from i–1 down to 1.

4. Algorithm

1. Start.
2. Read the number of rows.
3. If rows are outside 1 to 9, display an error.
4. For each row i, print leading spaces, ascending numbers from 1 to i, and descending numbers from i–1 to 1.
5. Stop.

5. Flowchart

Start → Read rows → Valid range? → Nested loops build each row → Print pattern → End

6. Pseudocode

READ rows\nIF rows < 1 OR rows > 9: DISPLAY invalid\nELSE\n FOR i = 1 TO rows\n PRINT spaces for alignment\n FOR j = 1 TO i: PRINT j without newline\n FOR j = i-1 DOWN TO 1: PRINT j without newline\n PRINT newline\nSTOP

7. Python Source Code

```
rows = int(input("Enter number of rows (1-9): "))

if rows < 1 or rows > 9:
    print("Invalid input. Enter a number from 1 to 9.")
else:
    for i in range(1, rows + 1):
        print(" " * (rows - i), end="")
        for j in range(1, i + 1):
            print(j, end="")
        for j in range(i - 1, 0, -1):
            print(j, end="")
        print()
```

8. Test Cases

Input	Expected check	Expected result
rows=3	pattern	1\n 121\n12321
rows=0	invalid	Invalid input. Enter a number from 1 to 9.

9. Sample Output Screenshot / Transcript

```
Enter number of rows (1-9): 4
   1
  121
 12321
1234321
```

10. Conclusion

Nested loops control spaces and digits to generate a symmetrical numeric pattern.

Case Study 29: Password Policy Validation Engine

1. Problem Statement

Check a sample password against simple rules and display every rule it fails.

2. Objectives

• Use string operations and loops.
• Check length and character categories.
• Report multiple validation errors clearly.

3. Assumptions

• A valid password has at least 8 characters, one uppercase letter, one lowercase letter, one digit, and one special character from !@#$%&*.
• This is a classroom exercise and not a complete real-world security checker.

4. Algorithm

1. Start.
2. Read a password.
3. Check its length.
4. Loop through its characters to identify uppercase, lowercase, digit, and special characters.
5. Collect any failed rules.
6. Display valid or list failed rules.
7. Stop.

5. Flowchart

Start → Read password → Check length and character types → Any failed rules? → Display result → End

6. Pseudocode

READ password\nSET flags for upper, lower, digit, special to false\nFOR each character in password\n UPDATE matching flags\nIF length < 8: add length error\nIF any flag is false: add corresponding error\nIF no errors: DISPLAY valid ELSE DISPLAY errors\nSTOP

7. Python Source Code

```
password = input("Enter a sample password: ")
has_upper = False
has_lower = False
has_digit = False
has_special = False
special_chars = "!@#$%&*"
errors = []

for char in password:
    if char.isupper():
        has_upper = True
    if char.islower():
        has_lower = True
    if char.isdigit():
        has_digit = True
    if char in special_chars:
        has_special = True

if len(password) < 8:
    errors.append("Use at least 8 characters.")
if not has_upper:
    errors.append("Include an uppercase letter.")
if not has_lower:
    errors.append("Include a lowercase letter.")
if not has_digit:
    errors.append("Include a digit.")
if not has_special:
    errors.append("Include a special character from !@#$%&*.")

if len(errors) == 0:
    print("Password meets the classroom policy.")
else:
    print("Password does not meet the policy:")
    for error in errors:
        print("-", error)
```

8. Test Cases

Input	Expected check	Expected result
Study@123	all rules met	Password meets the classroom policy.
hello	short; no uppercase, digit, special	four relevant errors displayed

9. Sample Output Screenshot / Transcript

```
Enter a sample password: Study@123
Password meets the classroom policy.
```

10. Conclusion

The program practises string traversal and Boolean flags to check multiple password rules.

Case Study 42: Student Marks Matrix Analysis

1. Problem Statement

Store marks for several students across several subjects in a nested list, calculate each student's total and average, and calculate the average for each subject.

2. Objectives

• Represent tabular data using nested lists.
• Calculate row-wise and column-wise statistics.
• Identify the student with the highest total.

3. Assumptions

• All marks are between 0 and 100.
• Every student has a mark for every subject.
• Student and subject names are entered in the same order as the marks.

4. Algorithm

1. Start.
2. Read student and subject counts and their names.
3. Use nested loops to collect marks into a matrix.
4. Calculate each student's total and average.
5. Calculate each subject's average using column values.
6. Find the student with the highest total and display the results.
7. Stop.

5. Flowchart

Start → Read names and dimensions → Fill nested-list marks matrix → Calculate row totals and column averages → Find highest total → Display report → End

6. Pseudocode

READ student_count, subject_count\nREAD student names and subject names\nFOR each student i\n FOR each subject j: READ marks[i][j]\nFOR each student: calculate total and average\nFOR each subject j: calculate average of marks in column j\nFIND index of maximum student total\nDISPLAY all results\nSTOP

7. Python Source Code

```
student_count = int(input("Number of students: "))
subject_count = int(input("Number of subjects: "))

if student_count <= 0 or subject_count <= 0:
    print("Counts must be positive.")
else:
    students = []
    subjects = []
    for i in range(student_count):
        students.append(input("Student name: "))
    for j in range(subject_count):
        subjects.append(input("Subject name: "))

    marks = []
    valid = True
    for i in range(student_count):
        row = []
        print("Marks for", students[i])
        for j in range(subject_count):
            mark = float(input(subjects[j] + ": "))
            if mark < 0 or mark > 100:
                valid = False
            row.append(mark)
        marks.append(row)

    if not valid:
        print("Invalid marks. Each mark must be from 0 to 100.")
    else:
        totals = []
        for i in range(student_count):
            total = sum(marks[i])
            totals.append(total)
            average = total / subject_count
            print(students[i], "- total:", total, "average:", round(average, 2))
```

```
for j in range(subject_count):
    subject_total = 0
    for i in range(student_count):
        subject_total += marks[i][j]
    print(subjects[j], "average:", round(subject_total / student_count, 2))

top_index = totals.index(max(totals))
print("Highest total:", students[top_index], "-", max(totals))
```

8. Test Cases

Input	Expected check	Expected result
A: 80,90; B: 70,80	A total 170 avg 85; B total 150 avg 75	Subject averages 75 and 85; A highest
mark=120	invalid	Invalid marks message

9. Sample Output Screenshot / Transcript

```
Number of students: 2
Number of subjects: 2
Student name: Asha
Student name: Ravi
Subject name: Python
Subject name: Maths
Marks for Asha
Python: 80
Maths: 90
Marks for Ravi
Python: 70
Maths: 80
Asha - total: 170.0 average: 85.0
Ravi - total: 150.0 average: 75.0
Python average: 75.0
Maths average: 85.0
Highest total: Asha - 170.0
```

10. Conclusion

The nested-list matrix supports both student-wise and subject-wise calculations.

Case Study 49: Tuple-Based Product Catalogue Audit

1. Problem Statement

Store product identifier, price, and stock quantity in tuples; calculate inventory value and identify products that need replenishment.

2. Objectives

• Represent fixed product records using tuples.
• Use loops and conditions to validate records.
• Calculate inventory value and flag low stock.

3. Assumptions

• Each product tuple contains (identifier, price, quantity).
• Price and quantity must be non-negative.
• A product with fewer than 5 units is marked for replenishment.

4. Algorithm

1. Start.
2. Create a list of product tuples.
3. For each tuple, validate price and quantity and calculate price × quantity.
4. Add valid inventory value to the total.
5. Display products with stock below 5.
6. Display total inventory value.
7. Stop.

5. Flowchart

Start → Create product tuples → Loop through records → Validate → Calculate stock value → Check low stock → Print total → End

6. Pseudocode

products = list of tuples (id, price, quantity)\ntotal_value = 0\nFOR each product tuple\n UNPACK id, price, quantity\n IF price < 0 OR quantity < 0: DISPLAY invalid\n ELSE\n total_value = total_value + price * quantity\n IF quantity < 5: DISPLAY replenishment needed\nDISPLAY total_value\nSTOP

7. Python Source Code

```
products = [
    ("P101", 25.0, 12),
    ("P102", 80.0, 3),
    ("P103", 15.0, 0)
]

total_value = 0.0
print("Product catalogue audit")
for product in products:
    product_id, price, quantity = product
    if price < 0 or quantity < 0:
        print(product_id, "- invalid record")
    else:
        value = price * quantity
        total_value += value
        print(product_id, "- stock value: ■{:.2f}".format(value))
        if quantity < 5:
            print("  Replenishment required (stock below 5).")

print("Total inventory value: ■{:.2f}".format(total_value))
```

8. Test Cases

Input	Expected check	Expected result
P101 price 25 qty 12	value 300	No replenishment
P102 price 80 qty 3	value 240	Replenishment required
P103 price 15 qty 0	value 0	Replenishment required

9. Sample Output Screenshot / Transcript

```
Product catalogue audit
P101 - stock value: ■300.00
P102 - stock value: ■240.00
  Replenishment required (stock below 5).
P103 - stock value: ■0.00
  Replenishment required (stock below 5).
Total inventory value: ■540.00
```

10. Conclusion

Tuples and tuple unpacking provide a simple way to audit fixed product records and find low-stock items.

Case Study 53: Food Waste Monitoring System

1. Problem Statement

For several days, compare meals prepared with meals served, calculate leftover meals and waste percentage, and flag days above a chosen waste limit.

2. Objectives

• Use loops to process daily figures.
• Calculate waste percentage using arithmetic.
• Use a condition to identify excessive waste.

3. Assumptions

• Meals prepared must be greater than zero.
• Meals served must be between zero and meals prepared.
• A waste percentage above 15% is considered excessive.

4. Algorithm

1. Start.
2. Read the number of days.
3. For each day, read meals prepared and served.
4. Validate the counts.
5. Calculate leftovers and waste percentage.
6. Flag days where waste exceeds 15%.
7. Stop.

5. Flowchart

Start → Read number of days → Loop: read prepared and served → Validate → Calculate leftovers and percentage → Compare with 15% → Display daily result → End

6. Pseudocode

READ days\nREPEAT for each day\n READ prepared, served\n IF prepared <= 0 OR served < 0 OR served > prepared: DISPLAY invalid\n ELSE\n leftovers = prepared - served\n waste_percent = leftovers / prepared * 100\n DISPLAY leftovers and waste_percent\n IF waste_percent > 15: DISPLAY excessive waste\nSTOP

7. Python Source Code

```
days = int(input("Number of days: "))

if days <= 0:
    print("Enter at least one day.")
else:
    for day in range(1, days + 1):
        prepared = int(input("Day " + str(day) + " meals prepared: "))
        served = int(input("Day " + str(day) + " meals served: "))

        if prepared <= 0 or served < 0 or served > prepared:
            print("Invalid meal counts.")
            continue

        leftovers = prepared - served
        waste_percent = leftovers / prepared * 100
        print("Leftovers:", leftovers)
        print("Waste: {:.2f}%".format(waste_percent))
        if waste_percent > 15:
            print("Warning: excessive food waste.")
        else:
            print("Waste is within the acceptable limit.")
```

8. Test Cases

Input	Expected check	Expected result
prepared=100, served=80	20 leftovers, 20%	Excessive waste warning
prepared=100, served=90	10 leftovers, 10%	Within acceptable limit

9. Sample Output Screenshot / Transcript

```
Number of days: 1
```

```
Day 1 meals prepared: 100
Day 1 meals served: 80
Leftovers: 20
Waste: 20.00%
Warning: excessive food waste.
```

10. Conclusion

The program calculates daily food waste and identifies days that exceed the assumed 15% limit.